

Ripetizioni Francesco

Marini Mattia

2025

Ripetizioni Francesco is licensed under [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/) .

© 2023 [Mattia Marini](#)

Indice

1	Java	3
1.1	Aspetti simili a c++	3
1.1.1	Dichiarazione delle variabili:	3
1.1.2	Commenti:	3
1.1.3	Cicli:	3
1.1.4	Operatori	4
1.1.5	If e switch	4
1.1.6	Funzioni	4
1.2	Differenze	4
1.2.1	Stampa su terminale	4
1.2.2	Linguaggio interpretato vs compilato	5
1.2.3	Memoria e puntatori	5
1.2.4	Object orientation	6
1.3	Esempio programma	6
1.4	Funzioni utili	6
1.4.1	Stringhe	7
1.4.2	Vettori	8
1.5	Parti utili della libreria standard	8
1.5.1	Math	9
1.5.2	ArrayList	9
1.5.3	Scanner	9
1.5.4	Random	10
1.5.5	LinkedList	10
1.5.6	Altre strutture dati utili	10
1.6	Lettura/scrittura su file	11
1.7	Try catch ed eccezioni	11
1.7.1	Checked vs unchecked	11
1.8	Scanner	12
1.9	File	13
1.10	Esercizi	13
1.11	Cheatsheet	20

Esercizi

1	Hello World (hello_world)	13
2	Hello classe (estrazione)	14
3	Geometria 1 (geometry1)	14
4	Geometria 1 (geometry1)	15
5	Montecarlo (montecarlo)	15
6	Sistema gestione prenotazioni centro sportivo	17
7	ASCII art	18
8	Operazioni arraylist	19

1 Java

Java è molto simile dal punto di vista della sintassi al c++. Non sarà molto complicato il passaggio

1.1 Aspetti simili a c++

La sintassi di Java è molto simile a quella di c++, ecco gli aspetti che rimangono invariati o quasi:

1.1.1 Dichiarazione delle variabili:

Sintassi	Differenze rispetto al c++
<code>int x = 15</code>	invariato
<code>long x = 15</code>	invariato
<code>float x = 15.0f</code>	nota il 15.0f, dove f sta per float
<code>double x = 15.0</code>	float ma con precision maggiore, 64 bit
<code>boolean x = true</code>	boolean anzichè bool
<code>String s = "stringa"</code>	dato String sarebbe una classe ma è trattato come tipo primitivo, dato che è usato molto frequentemente
<code>String v[] = new String[15]</code>	vettore di stringhe di dimensione 15
<code>int v[] = new int[15]</code>	vettore di interi di dimensione 15

1.1.2 Commenti:

- Commento riga singola: `// commento`
- Commento righe multiple: `/* commento */`

1.1.3 Cicli:

- Ciclo for:

```
for(int i = 0; i<15; i++){  
    // qualcosa  
}
```

- Ciclo while:

```
while(i < 15){  
    // qualcosa  
}
```

1.1.4 Operatori

Operatore	Descrizione
+ - * /	operatori matematici
&&	and logico
	or logico
!	not logico

1.1.5 If e switch

```
if(a<b) {
    //codice
}
else(if a>b) {
    //codice
}
else {
    //codice
}

switch(espressione) {
    case x:
        // codice
        break;
    case y:
        // codice
        break;
    default:
        // codice
}
```

1.1.6 Funzioni

```
void nome_funzione1 (int arg_1, String arg_2){
    //corpo funzione
}

int nome_funzione2 (int arg_1, String arg_2){
    //corpo funzione
    return 5;
}
```

Una funzione è quindi definita indicando nel seguente ordine, esattamente come in c++:

1. Tipo di ritorno (o void se non ritorna nulla)
2. Nome della funzione
3. Parametri, racchiusi fra parentesi tonde e separati da virgole
4. Corpo della funzione fra graffe

1.2 Differenze

1.2.1 Stampa su terminale

Una delle feature usata moltissimo, ma completamente diversa dal c++ è la stampa su terminale:

```

System.out.println("Stampa questa cosa");
//stampa andando a capo prima di stampare

System.out.print("Stampa questa cosa");
//stampa SENZA andando a capo

```

nota che le stringhe si possono concatenare con l'operatore +:

```

String a = "Hello";
String b = "world";

System.out.println(a + b);
//stampa "Hello world"

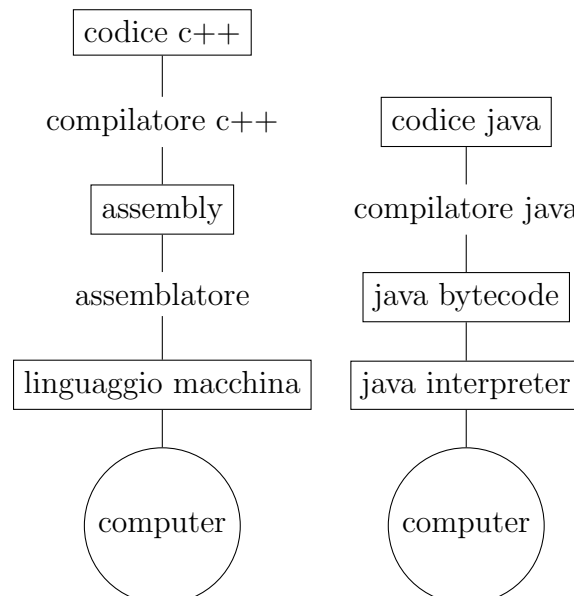
String c = a + b;
//Inizializza c a "Hello world"

```

1.2.2 Linguaggio interpretato vs compilato

A differenza di c++, java è un linguaggio interpretato

- *Linguaggio compilato*: il codice è "dato in pasto" a un compilatore, il quale lo converte in linguaggio macchina (di fatto in una sequenza di, 0 ed 1)
- *Linguaggio interpretato*: il codice è "dato in pasto" ad un compilatore, il quale lo converte però in bytecode, ossia un linguaggio di basso livello (molto difficile da leggere e scrivere), il quale è in grado di essere letto da un *interprete*



1.2.3 Memoria e puntatori

Java è un linguaggio ad alto livello che gestisce la memoria in maniera diversa rispetto al c++:

- c++: il compito di allocare e deallocare la memoria non più utilizzata è del programmatore
- java: la memoria viene deallocata in maniera automatica tramite un meccanismo chiamato garbage collection

Visto che in java la memoria è gestita in maniera automatica, il programmatore non ne ha accesso diretto tramite puntatori: al contrario, i puntatori non esistono

1.2.4 Object orientation

Sebbene c++ sia un linguaggio che permette di utilizzare classi ed oggetti, in java l'object orientation è forzata: ogni parte del programma deve essere contenuta all'interno di una classe

1.3 Esempio programma

```
class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Cose da notare:

- La funzione main è contenuta all'interno di una classe "HelloWorld", il cui nome è arbitrario
- La funzione main è marcata come **static**, ciò vuol dire che la funzione esiste anche se non esiste un oggetto di tipo "HelloWorld", affronteremo meglio il modificatore **static** più avanti, per ora possiamo ignorarlo
- La funzione main è marcata come **public**, ciò vuol dire che la funzione è accessibile ovunque. Affronteremo meglio questo modificatore più avanti, per ora possiamo ignorarlo
- La funzione main prende in ingresso un vettore di stringhe. Nel caso si avviasse l'applicazione da terminale è possibile passare al main dei parametri nel seguente modo:

```
cd cartella_applicazione
./nome_applicazione parametro_1 parametro_2 ...
```

in questo caso il vettore di stringhe **args** conterrà **parametro_1** e **parametro_2**. Penso non lo userete mai ma è buono saperlo

1.4 Funzioni utili

In java sono definite alcune funzioni utilissime. Qui una lista (non esaustiva) delle più comuni:

1.4.1 Stringhe

Supponiamo di avere `String s = "stringa stringa";`

Funzione	Descrizione
<code>s.length()</code>	Ritorna la lunghezza della stringa
<code>s.charAt(int index)</code>	Ritorna il carattere in posizione <code>index</code>
<code>s.replace(char c1, char c2)</code>	Sostituisce tutte le occorrenze di <code>c1</code> con <code>c2</code>
<code>s.equals(String str)</code>	Verifica se due stringhe sono uguali
<code>s.equalsIgnoreCase(String str)</code>	Come sopra, ignorando maiuscole/minuscole
<code>s.compareTo(String str)</code>	Confronto lessicografico (ordine alfabetico)
<code>s.toCharArray()</code>	Converte in un array di caratteri <code>char[]</code>

Tabella 1: Funzioni basi

Funzione	Descrizione
<code>s.indexOf(char carattere)</code>	Ritorna l'indice della prima occorrenza di <code>carattere</code> in <code>s</code>
<code>s.indexOf(String stringa)</code>	Ritorna l'indice della prima occorrenza della sottostringa <code>stringa</code> in <code>s</code>
<code>s.lastIndexOf(String str)</code>	Ritorna l'indice dell'ultima occorrenza della sottostringa <code>str</code> in <code>s</code>
<code>s.contains(CharSequence seq)</code>	Ritorna <code>true</code> se la sequenza <code>seq</code> è contenuta in <code>s</code>
<code>s.startsWith(String prefix)</code>	Verifica se <code>s</code> inizia con la stringa indicata
<code>s.endsWith(String suffix)</code>	Verifica se <code>s</code> termina con la stringa indicata

Tabella 2: Funzioni di ricerca in stringhe

Funzione	Descrizione
<code>s.substring(int beginIndex)</code>	Restituisce la sottostringa da <code>beginIndex</code> fino alla fine
<code>s.substring(int begin, int end)</code>	Restituisce la sottostringa da <code>begin</code> (incluso) a <code>end</code> (escluso)
<code>s.split(String regex)</code>	Divide la stringa in un array, usando l'espressione regolare come separatore
<code>String.join(delim, ...)</code>	Unisce più stringhe usando il delimitatore specificato
<code>s.trim()</code>	Rimuove gli spazi (whitespace) iniziali e finali dalla stringa
<code>s.strip()</code> (Java 11+)	Rimuove spazi secondo le regole Unicode

Tabella 3: Funzioni di splitting

Funzione	Descrizione
<code>s.replaceAll(String regex, String replacement)</code>	Sostituisce tutte le occorrenze che corrispondono all'espressione regolare con un'altra stringa
<code>s.replaceFirst(String regex, String replacement)</code>	Sostituisce solo la prima occorrenza (secondo regex)
<code>s.toUpperCase()</code>	Converte tutti i caratteri in maiuscolo
<code>s.toLowerCase()</code>	Converte tutti i caratteri in minuscolo

Tabella 4: Funzioni di modifica

```
String s = "stringa stringa";
s.length(); // 15
s.charAt(2); // 'r'
s.indexOf('r'); // 2
s.indexOf("ga"); // 5
```

1.4.2 Vettori

Supponiamo di avere `int v[] = new int[15];`

Funzione	Descrizione
<code>v.length</code>	ritorna il numero di elemnti contenuti nel vettore

1.5 Parti utili della libreria standard

La libreria standard di java offre moltissime classi utili. Vediamo qui le più comuni

1.5.1 Math

Funzione	Descrizione
<code>Math.exp(float n)</code>	Ritorna e^n
<code>Math.log(float n)</code>	Ritorna $\ln(n)$
<code>Math.abs(float x)</code>	Ritorna $ x $ (valore assoluto di x)
<code>Math.sin(float x)</code>	Ritorna $\sin(x)$
<code>Math.cos(float x)</code>	Ritorna $\cos(x)$
<code>Math.tan(float x)</code>	Ritorna $\tan(x)$
<code>Math.asin(float x)</code>	Ritorna $\arcsin(x)$
<code>Math.acos(float x)</code>	Ritorna $\arccos(x)$
<code>Math.atan(float x)</code>	Ritorna $\arctan(x)$
<code>Math.max(float a, float b)</code>	Ritorna l'elemento maggiore fra a e b
<code>Math.min(float a, float b)</code>	Ritorna l'elemento minore fra a e b
<code>Math.floor(float x)</code>	Arrotonda per difetto x
<code>Math.ceil(float x)</code>	Arrotonda per eccesso x
<code>Math.round(float x)</code>	Arrotonda x

1.5.2 ArrayList

Utile per avere un vettore di lunghezza variabile. Si inizializza nel seguente modo:

```
ArrayList< tipo > nomeArray = new ArrayList< tipo >(dimensione)
```

Nota che:

- `dimensione` si può omettere, ottenendo un vettore con dimensione nulla
- `tipo` deve essere una classe. Se voglio un `ArrayList` di tipi primitivi devo utilizzare le classi wrapper (`Integer`, `Boolean`, `Double` ...)

Metodo	Descrizione
<code>v.get(index)</code>	ritorna l'elemento all'indice <code>index</code>
<code>v.set(index , element)</code>	setta l'elemento a indice <code>index</code>
<code>v.add(element)</code>	inserisce <code>element</code> in fondo
<code>v.remove (index)</code>	rimuove l'elemento a indice <code>index</code>
<code>v.size()</code>	ritorna il numero di elementi contenuti
<code>v.clear()</code>	rimuove tutti gli elementi

Inoltre, sono disponibili features di java per inizializzare `ArrayList` in maniera conveniente:

```
ArrayList<Integer> lista = new ArrayList<>(Arrays.asList(1,2,3));
```

1.5.3 Scanner

La classe `Scanner` ci permette di leggere input utente da terminale (e anche da file, ma non ci servirà). Uno `Scanner` si inizializza così:

```
Scanner nomeScanner = new Scanner(System.in);
```

Per leggere l'input da terminale ci sono i seguenti comandi:

Metodo	Descrizione
<code>nextBoolean()</code>	Legge un boolean da terminale
<code>nextByte()</code>	Legge un byte da terminale
<code>nextDouble()</code>	Legge un double da terminale
<code>nextFloat ()</code>	Legge un float da terminale
<code>nextInt()</code>	Legge un int da terminale
<code>nextLine()</code>	Legge una String da terminale
<code>nextLong()</code>	Legge un long da terminale
<code>nextShort()</code>	Legge uno short da terminale

Il metodo più comune è `nextLine()`, dato che ci restituisce l'intera riga come stringa, anche se contiene numeri

1.5.4 [Random](#)

Random fornisce un modo comodo per generare numeri casuali. Inizializza con

```
Random nome = new Random()
```

Metodo	Descrizione
<code>nextInt(range)</code>	genera un numero casuale nel range [0, range)
<code>nextFloat()</code>	genera un float in range [0.0, 1.0]
<code>nextDouble()</code>	genera un double in range [0.0, 1.0]

1.5.5 [LinkedList](#)

Lista linkata. Utilizzabile in modo molto simile al `ArrayList`. L'accesso agli elementi è molto più lento, rimozioni inserimenti sono molto più veloci

1.5.6 [Altre strutture dati utili](#)

- `HashSet`: insieme matematico, possibile vedere se un elemento è contenuto in esso in maniera efficiente
- `Map`: utile poter collegare ad ogni valore un altro valore detto chiave. Si può risalire al valore tramite la chiave in maniera efficiente
- `Stack`
- `Queue`
- `PriorityQueue`: struttura nella quale è possibile accedere all'elemento maggiore in maniera efficiente
- `SortedSet`: struttura nella quale i dati mantengono sempre un ordinamento crescente. Non ammette duplicati

1.6 Lettura/scrittura su file

1.7 Try catch ed eccezioni

In java e in molto linguaggi di programmazione moderni vengono forniti meccanismi standard per gestire situazioni inaspettate che non possono però essere previste con certezza (es. un file non esiste, l'utente ha inserito un dato non valido etc etc). Per queste situazioni esistono oggetti dette *eccezioni*, fatte per descrivere l'errore che si è verificato.

L'idea generale è che qualunque pezzo di codice può sollevare o tirare un'eccezione e, nel caso in cui questa non venga gestita (ossia circondata da un blocco try catch nella porzione di programma chiamante), allora l'applicazione termina, stampando una descrizione della eccezione avvenuta

Il codice che tira eccezioni va circondato da un blocco try catch come segue:

```
try {
    // codice che potrebbe sollevare un'eccezione
} catch (TipoEccezione1 e1) {
    // codice per gestire l'eccezione di tipo TipoEccezione1
} catch (TipoEccezione2 e2) {
    // codice per gestire l'eccezione di tipo TipoEccezione2
} finally {
    // codice che viene eseguito sempre, sia che si sia verificata
    un'eccezione sia che non si sia verificata
}
```

Ad esempio, se avessimo

```
try {
    f1() // solleva eccezione TipoEccezione2
} catch (TipoEccezione1 e1) {
    System.out.println("e1");
} catch (TipoEccezione2 e2) {
    System.out.println("e2");
} finally {
    System.out.println("finally");
}
// stampa:
// e2
// finally
```

Se invece facessimo qualcosa di questo tipo all'interno del main:

```
f1() // solleva eccezione TipoEccezione2
```

allora il programma si interromperebbe stampando informazioni riguardo l'eccezione sollevata

1.7.1 Checked vs unchecked

Esistono due tipi di eccezioni: *checked* e *unchecked*. Dal punto di vista logico le prime rappresentano tutte le condizioni che non NON abbiamo modo di verificare non accadano, mentre nel secondo, utilizzando determinate accortezze possiamo evitare succedano. Per questa ragione, siamo *obbligati* a gestire le prime, mentre possiamo ignorare le seconde.

Checked exceptions	Unchecked exceptions
IOException	ArithmeticException
ClassNotFoundException	NullPointerException
FileNotFoundException	ArrayIndexOutOfBoundsException
SQLException	IllegalArgumentException

L'idea è che le unchecked exceptions possono essere ignorate proprio in virtù del fatto che tramite determinate accortezze possiamo evitare che si verifichino (es. controllare che un indice sia valido prima di accedere ad un array, controllare che un divisore non sia 0 prima di effettuare una divisione etc etc), mentre ciò non è vero per quanto riguarda le checked

Ciò che cambia nel pratico è che quando utilizziamo una porzione di codice che può sollevare una checked exception, il compilatore ci obbliga a circondarla con un blocco try catch, oppure a dichiarare che la funzione che la contiene può sollevare quell'eccezione (usando la parola chiave `throws`):

```
public f1() throws CheckedException{}
```

Si noti che in realtà è possibile dichiarare una funzione con la keyword `throw` anche per le unchecked exceptions, ma non è obbligatorio

1.8 Scanner

Scanner fornisce il modo più semplice (ma più lento) per leggere da file o da `stdin`:

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

try (Scanner scanner = new Scanner(new File("path/to/file.txt"))) {
    while (scanner.hasNextLine()) {
        String line = scanner.nextLine();
        // Process the line
    }
} catch (FileNotFoundException e) {
    e.printStackTrace();
}
```

per la scrittura invece il metodo preferito è il seguente:

```
import java.io.PrintWriter;
import java.io.FileWriter;
import java.io.IOException;

try (PrintWriter writer = new PrintWriter(new FileWriter("esempio.txt",
    true))) {
    writer.println("Riga aggiunta!");
} catch (IOException e) {
    e.printStackTrace();
}
```

1.9 File

Il package `nio.file` un modo più rapido e moderno per leggere da file:

```
package com.mycompany.app;

import java.nio.file.Files;
import java.nio.file.Paths;
import java.io.IOException;

String content=new
    String(Files.readAllBytes(Paths.get("path/to/file.txt")));
List<String>lines=Files.readAllLines(Paths.get("path/to/file.txt"));
```

per la scrittura invece il metodo preferito è il seguente:

```
import java.nio.file.Files;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.io.IOException;

public class AppendFileNio {
    public static void main(String[] args) {
        String nuovaRiga = "Nuova riga aggiunta!";
        try {
            Files.write(
                Paths.get("esempio.txt"),
                Arrays.asList(nuovaRiga),
                StandardOpenOption.APPEND
            );
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

1.10 Esercizi

Collezione di esercizi, divisi per categoria

Esercizio 1: *Hello World (hello_world)*

Scrivere un programma in Java che prenda in input una stringa `s` e un numero `n` e stampi `s` `n` volte, accompagnata dal numero di riga. Ad esempio, dati in input `userin` e `4`, l'output sarà:

1. userin
2. userin
3. userin
4. userin

Esercizio 2: *Hello classe (estrazione)*

Crea un programma per giocare a una sorta di roulette modificata. Le regole sono le seguenti:

- Vengono estratte delle palline che hanno un colore (**rosso**, **giallo** o **verde**) e un numero da 1 a 9 estremi compresi
- Il giocatore deve indicare un numero e un colore
- Il punteggio viene assegnato così:
 - Colore giusto: **+1**
 - Un punteggio che varia da 0 a 4 in base alla distanza del numero previsto da quello estratto, ad esempio, detto n_e il numero *estratto* e n_p il numero *previsto*

$$\left(1 - \frac{|n_e - n_p|}{8}\right) \cdot 4$$

- Se colore e numero sono uguali: **+1**

Il programma deve chiedere in input all'utente un numero e un colore ed estrarre una pallina casualmente, per poi stampare in output il punteggio ottenuto. Si usi una classe **ball** e una classe **game** per gestire le partite.

Il programma deve partire chiedendo all'utente il numero n di round che vuole giocare. Dopodiché, verranno chieste n previsioni all'utente; per ognuna deve essere stampato il punteggio e alla fine degli n round deve essere visualizzato il punteggio totale

Esercizio 3: *Geometria 1 (geometry1)*

Creare una classe per ciascuno dei seguenti oggetti geometrici: **Punto**, **Rettangolo**, **Cerchio**. Creare una classe **PianoCartesiano** che possa contenere le tre classi elencate precedentemente. Una volta inserita una classe nel piano cartesiano verrà ritornato un **id**, tramite il quale possiamo accedervi (l'istanza dell'oggetto aggiunto rimane scollegata da quella inserita nel piano cartesiano). Creare un metodo **closer** che prenda un **id** esistente e ritorni l'id della forma geometrica più vicina (usare il centro delle figure per confrontare la distanza)

Esercizio 4: *Geometria 1 (geometry1)*

Creare una classe per ciascuno dei seguenti oggetti geometrici: **Punto**, **Rettangolo**, **Cerchio**. Creare una classe **PianoCartesiano** che possa contenere le tre classi elencate precedentemente. Una volta inserita una classe nel piano cartesiano verrà ritornato un **id**, tramite il quale possiamo accedervi (l'istanza dell'oggetto aggiunto rimane scollegata da quella inserita nel piano cartesiano). Creare un metodo **closer** che prenda un **id** esistente e ritorni l'id della forma geometrica più vicina (usare il centro delle figure per confrontare la distanza)

Esercizio 5: Montecarlo (*montecarlo*)

Il casinò di Montecarlo ha richiesto una nuova slot machine; lo scopo del gioco è quello di simulare la pesca di *cinque carte* da un mazzo standard (con quattro semi, le carte da 2 a 10, l'asso(1) le tre figure (11,12,13) e niente jolly)

Le opportunità di vincita (con quattro delle 5 carte estratte) sono le seguenti:

Combinazione	vincita
Poker di figure	1000 €
Poker d'assi	2000 €
^a Colore	Somma delle carte uscite $\times 8$
^b Scala colore	Somma delle carte uscite $\times 10$

La presenza di un'altra carta non intacca la vincita.

Contare quante slotmachine sono presenti nel casino

Si scrivano le classi opportune per implementare il gioco e un classe **Casino** dove create diverse **Slot**.

Fare in modo che la giocata possa essere effettuata solo dopo aver inserito un certo importo;

Introdurre successivamente anche i *jolly* tra le carte e fare in modo che la presenza del jolly come carta estratta raddoppi la vincita;

^a5 carte dello stesso seme, ad es: 1-4-5-10-13, *tutti cuori* $\rightarrow$ vincita = 264 €

^b5 carte dello stesso seme in sequenza, ad esempio: 3-4-5-6-7 *tutte picche* $\rightarrow$ vincita = 250 €

Esercizio 6: Sistema gestione prenotazioni centro sportivo

Il Centro Sportivo *ActiveWorld* offre diverse attività (*nuoto, palestra, yoga, arrampicata*) e vuole un software che gestisca le prenotazioni giornaliere per i vari corsi e spazi. Tutte le informazioni devono essere salvate e gestite tramite file di testo, letti all'avvio dell'applicazione e aggiornati durante l'esecuzione.

File coinvolti: l'applicazione utilizza tre file:

- *Utenti registrati:* ogni utente è caratterizzato da:

- nome
- cognome
- data di nascita
- telefono
- email

- *Attività disponibili*

- nome dell'attività

- capacità massima
- fascia oraria (es. 18:00--19:30)
- *Prenotazioni attive*: ogni riga del file contiene:
 - email dell'utente
 - nome attività
 - data della prenotazione (gg/mm/aaaa)

Funzionalità richieste

1. *Caricamento dati dai 3 file*: all'avvio, il programma deve:
 - leggere tutti gli utenti registrati
 - leggere tutte le attività disponibili
 - leggere le prenotazioni attive
 - verificare che ogni prenotazione faccia riferimento a un utente e a un'attività validi
 - segnalare eventuali incongruenze (es. utente inesistente → prenotazione ignorata)
2. *Visualizzare le attività disponibili*: mostrare
 - nome attività
 - fascia oraria
 - numero prenotazioni attuali
 - posti disponibili rimanenti
3. *Visualizzare tutte le prenotazioni*: raggruppare per attività, ordinate per data.
4. *Cercare le prenotazioni di un utente*: richiedere *email* e mostrare tutte le sue prenotazioni.
5. *Aggiungere una nuova prenotazione*: il programma deve:
 - chiedere: email utente, attività, data
 - verificare che l'utente esista
 - verificare che l'attività esista
 - controllare che la data non sia nel passato
 - controllare che l'utente non abbia già una prenotazione per la stessa attività nella stessa giornata
 - controllare che l'attività non sia al completo per quella data
 - se tutto ok:
 - aggiungere la prenotazione in memoria

– aggiungerla al file in modalità *append*

6. *Cancellare una prenotazione*

Richiedere:

- email utente
- nome attività
- data

Controllare che esista, poi:

- rimuoverla dall'elenco
- aggiornare il file riscrivendolo completamente

7. *Report automatico delle attività quasi piene*: il programma deve offrire una funzione *Genera Report* che:

- mostra tutte le attività che hanno meno del 10% dei posti liberi nella data selezionata
- genera anche una versione salvata in un file `report.txt` con formattazione ordinata

8. *Statistiche settimanali (opzionale)*: dato un range di date, il programma deve indicare:

- quale attività è stata prenotata di più
- quale di meno
- il numero totale di prenotazioni
- percentuale di riempimento media delle attività

Requisiti e vincoli tecnici

- Il formato dei file deve rimanere sempre uniforme.
- Se vengono inseriti dati mancanti o non validi, il programma deve richiederli di nuovo.
- Ogni modifica deve aggiornare coerentemente i file.
- L'applicazione deve funzionare finché il gestore non sceglie di uscire.

Esercizio 7: ASCII art

Creare una applicazione che, dato in input un file *in formato .ppm p3* crei una ascii art di quanto rappresentato dalla figura. Il main deve definire due parametri:

- inputFile il file .ppm da convertire in ascii art
- outputFile il file .txt di output contenente la ascii art

Il formato .ppm p3 rappresenta una immagine tramite una matrice di pixel, ognuno dei quali è rappresentato da tre numeri interi (rosso, verde, blu) che vanno da 0 a 255. Il file inizia con una intestazione composta da:

```
P3
<WIDTH> <HEIGHT>
<MAX_COLOR>
<R> <G> <B> ... <R> <G> <B>
```

Ad esempio, un file contenente 3 pixel in orizzontale, in ordine, uno rosso, uno verde e uno blu, sarebbe così codificato:

```
P3
3 1
255
255 0 0 0 255 0 0 0 255
```

dove

$$\underbrace{255\ 0\ 0}_\text{pixel 1}\ \underbrace{0\ 255\ 0}_\text{pixel 2}\ \underbrace{0\ 0\ 255}_\text{pixel 3}$$

hint: si può usare la seguente formula per convertire un pixel in bianco e nero:

```
double grayscale = 0.299 * r + 0.587 * g + 0.114 * b;
```

Esercizio 8: Operazioni arraylist

Scrivere un programma che chieda all'utente un numero **n** di interi da terminale. L'applicazione deve innanzitutto chiedere **n** all'utente e successivamente leggere **n** interi in un arraylist. Una volta letti tutti i numeri viene stampato un prompt che chiede quale operazione eseguire. Le opzioni sono le seguenti:

1. *Verifica numero*: viene chiesto un numero e l'applicazione deve verificare se il numero è presente nell'arraylist
2. *Rimuovi numero*: viene chiesto un numero e l'applicazione deve rimuovere *tutte* le occorrenze di quel numero nell'arraylist
3. *Sort*: chiede se vuole ordinare l'arraylist in ordine crescente o decrescente e stampa il risultato ordinato. Usare un algoritmo di sort scritto a mano, non quello implementato nelle classi standard!
4. *Check sort*: stampa **true** se il vettore è ordinato, **false** altrimenti
5. *Reverse*: inverte l'ordine del vettore e lo stampa
6. *Quit*: esce dall'applicazione

1.11 Cheatsheet

Files-lettura

```
// Leggo parola/intero/float uno per volta
Scanner scanner = new Scanner(new File("path/to/file.txt"));

// Leggo intero contenuto linea per linea in lista di stringhe
String content=Files.readString(Paths.get("path/to/file.txt"));

// Leggo intero contenuto linea per linea in lista di stringhe
List<String>lines=Files.readAllLines(Paths.get("path/to/file.txt"));
```

Files-scrittura

```
// Scrivo stringa e chiudo file immediatamente
Files.writeString(Paths.get("file.txt"), "contenuto", <StandardOpenOption>);

StandardOpenOption.TRUNCATE_EXISTING // se file esiste cancella contenuto
StandardOpenOption.APPEND // se file esiste scrive in fondo

// Scrivo lista di stringhe
String array[] = { "line1", "line2" }; // va convertito con Arrays.asList()
ArrayList<String> arrayList = new ArrayList<>(Arrays.asList("line1", "line2"));

Files.write(Paths.get("file.txt"), Arrays.asList(array), <StandardOpenOption>);
Files.write(Paths.get("file.txt"), arrayList, <StandardOpenOption>);
```

Features Eclipse

Operazione	Menu	Scorciatoia
Go to definition (declaration)	Navigate → Open declaration	f3
Format	Source → Format	ctrl + shift + f
Automatically import libraries	Source → Organize imports	ctrl + shift + o
Rename	Refactor → Rename	alt + ctrl + r
Generate getters/setters	Source → Generate getters/setters	nessuna
Generate toString	Source → Generate toString	nessuna
Generate constructor	Source → Generate constructor using fields	nessuna

Date

```
LocalDate today = LocalDate.now(); // solo data
LocalDateTime now = LocalDateTime.now(); // data e ora

LocalDate someDay = LocalDate.of(2026, 11, 26);
LocalDateTime someTime = LocalDateTime.of(2026, 11, 26, 3, 30, 15);

// Conversione da stringa a data; formatter uguale per LocalDateTime e LocalDate
String date = "26/02/2026 15:30:45";
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm:ss");

LocalDateTime parsedDateTime = LocalDateTime.parse(date, formatter);
LocalDate parsedDate = LocalDate.parse(date, formatter);
```

Sort

```
ArrayList<SomeClass> list = new ArrayList<>();
list.sort((o1, o2) -> <criterio ordering>);
```

l'oggetto (o1, o2) -> <criterio ordering> serve per definire un ordinamento fra due istanze di SomeClass (o1, o2). <criterio ordinamento> deve essere un intero < 0 se $o_1 < o_2$, = 0 se $o_1 = o_2$ e > 0 se $o_1 > o_2$. Ad es, per ordinare rispetto al campo x di una classe: `list.sort((o1, o2) -> o1.x - o2.x)`

Enum con valori associati

```
enum Direction {
    UP("Su", 1), DOWN("Giù", 3), LEFT("Sinistra", 2), RIGHT("Destra", 4)
    private final String ita; private final int val;
    private Direction(String ita, int val) {
        this.ita = ita; this.val = val;
    }
}
```

Funzioni stringhe

```
// sostituzione
s.replace(char c1, char c2)
s.replaceFirst(String regex, String replacement)
s.replaceAll(String regex, String replacement),

// split e unione
s.trim(),
s.substring(int begin, int end)
s.split(String regex),
String.join(delim, ...)

// altre
s.compareTo(String str)
s.toUpperCase()
s.toLowerCase()
s.indexOf(String stringa)
```

Espressioni regolari

Wildcards	Quantificatori
.	Qualsiasi carattere
\s	Whitespace character
\d	Digit, cifra
\w	Word character

Regex	Descrizione
\w+\.\w+@(domain1 domain2 domainx)\.com	matcha "qualcosa.qualcosa@dominio.com"
\s*,\s*	matcha una virgola con spazi a sx e dx
\d{2}/\d{2}/\d{4}	matcha una data nel formato "gg/mm/aaaa"
https?://[\w-]+	matcha un URL HTTP o HTTPS
([A-Z][a-z]+)	matcha parole che iniziano con la maiuscola

```
import java.util.regex.Pattern;
import java.util.regex.Matcher;

Pattern pattern = Pattern.compile(<regex>);
Matcher matcher = pattern.matcher(<text>);

while (matcher.find()) { matcher.group(); }
```

Altro

```
// Ottenere path directory corrente
System.out.println(System.getProperty("user.dir"));
```